

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Грищук Елизавета Дмитриевна  
БР 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

## Задача

Реализовать REST API для приложения бронирования столиков в ресторанах на основе готового boilerplate. С учётом результатов ДЗ1 (проектирование базы данных) и ДЗ2 (проектирование эндпоинтов) реализовать модели, контроллеры и представления. Продемонстрировать полностью работающее API.

## Ход работы

В качестве основы был выбран boilerplate на базе Express + TypeORM, реализованный на TypeScript. Стек технологий: Node.js, Express, TypeORM, PostgreSQL, jsonwebtoken, bcryptjs, swagger-ui-express.

На основе спроектированной в ДЗ1 схемы базы данных были реализованы следующие модели (entities):

- User — пользователь системы (id, name, email, password\_hash)
- Category — категория кухни (id, name)
- Restaurant — ресторан (id, category\_id, name, address, price\_range, description, photo\_url)
- Table — столик (id, restaurant\_id, capacity)
- Booking — бронирование (id, user\_id, table\_id, booking\_time, guests\_count, status)
- Review — отзыв (id, user\_id, restaurant\_id, rating, comment)

Связи между сущностями реализованы через декораторы TypeORM: @ManyToOne, @OneToMany, @JoinColumn. Синхронизация схемы с базой данных осуществляется автоматически через параметр synchronize: true.

Аутентификация реализована посредством JWT-токенов. Пароли хешируются с использованием bcryptjs. Middleware authMiddleware проверяет наличие и валидность токена в заголовке Authorization: Bearer <token>.

На основе спроектированных в ДЗ2 эндпоинтов реализованы следующие контроллеры:

- AuthController — POST /auth/register, POST /auth/login
- UserController — GET /users/me, PATCH /users/me

- CategoryController — GET /categories
- RestaurantController — GET /restaurants, GET /restaurants/:id, GET /restaurants/:id/tables, GET /restaurants/:id/reviews, POST /restaurants/:id/reviews
- BookingController — GET /bookings, POST /bookings, GET /bookings/:id, PATCH /bookings/:id/cancel

Для каждого эндпоинта реализована обработка ошибок с возвратом соответствующих HTTP-статусов (400, 401, 403, 404, 409, 422). API задокументировано в формате OpenAPI 3.0 и доступно через Swagger UI по адресу /api/docs.

## **Вывод**

В ходе выполнения лабораторной работы был реализован полнофункциональный REST API для приложения бронирования столиков в ресторанах. Реализованы все 6 моделей и 13 эндпоинтов, спроектированных в ДЗ1 и ДЗ2. Реализована JWT-аутентификация, обработка ошибок и автоматическая документация через Swagger UI. API успешно протестировано в локальной среде.